МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего профессиональногообразования

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Инженерная школа информационных технологий и робототехники

Направление 15.03.06 «Мехатроника и робототехника»

Отделение автоматизации и робототехники

Знакомство с лабораторной базой. Изучение и настройка GPIO. Применение библиотеки CMSIS при программировании микроконтроллеров

Отчет по Лабораторной работе № 1

по дисциплине «Микропроцессоры и микроконтроллеры»

Вариант 22

Выполнил:
студент группы 8EM51 _____ _____ Степанов А.С.
Подпись Дата Фамилия И.О.

Проверил:
Старший преподаватель
(ОАР, ИШИТР)
_____ _____ Поберезкин Н.И.
Должность Подпись Дата Фамилия И.О.

Томск 2025

Содержание

Цель работы 3

Приведённое задание по варианту 3

Описание хода выполнения задания 3

Принципиальная схема подключения 4

Блок-схема алгоритма работы программы 5

Registers 6

Define 9

CMSIS 14

Графики внешних воздействий 18

Вывод 19

Томск 2025

Цель работы

Приобрести навыки взаимодействия с программной средой Visual Studio Code. Реализовать программный код на STM32 используя прямое обращение к памяти, собственные макросы и CMSIS.

Приведённое задание по варианту

Вариант 2

Собрать схему с включением 6-и светодиодов и 2-х кнопок. Первая кнопка задаёт количество одновременно включаемых светодиодов. Каждое нажатие второй кнопки включает следующее заданное количество светодиодов. Нажатие на вторую кнопку, после включения всех светодиодов, выключает их и процесс начинается сначала. В том случае, если первой кнопкой было задано одновременное включение 6-ти светодиодов, вторая кнопка просто включает все светодиоды и выключает их.

Описание хода выполнения задания

1) создать среду для выполнения программы;

2) составить блок-схему алгоритма работы программы;

3) изучить и обратиться непосредственно к reference manual RM0008 для stmf103c8t6;

4) написать код, используя регистры;

5) написать код, используя #define;

6) написать код, используя CMSIS;

7) построить графики в MCUViewer, отображающие работу программного кода;

8) сделать развёрнутый вывод по проделанной работе.

Томск 2025

Принципиальная схема подключения

Принципиальная схема подключения приведена в Приложении А.

Томск 2025

Блок-схема алгоритма работы программы

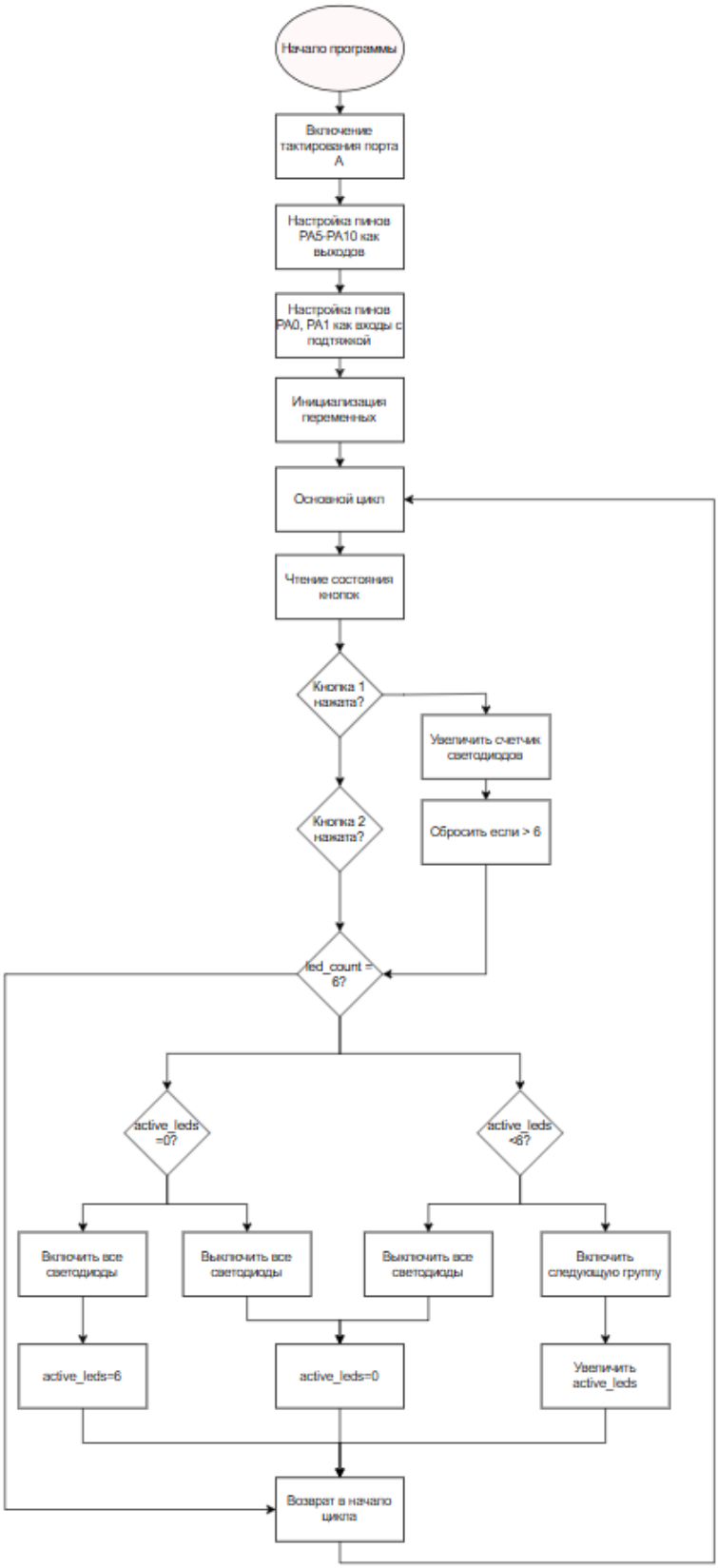


Рисунок 1 – Блок-схема алгоритма работы программы

Томск 2025

Registers

```c
#include <stdint.h>

uint8_t flag;
// Переменные
int led_count = 1;     // сколько включается одновременно (1..6)
int current_position = 0; // текущая позиция для следующей группы
uint8_t prev_btn1 = 1; // предыдущее состояние кнопок
uint8_t prev_btn2 = 1;

// Массив состояний ламп (0-выкл, 1-вкл)
uint8_t leds_state[6] = {0, 0, 0, 0, 0, 0};



void update_physical_leds(void) {
    // Синхронизируем физическое состояние с логическим
    for (int i = 0; i < 6; i++) {
        if (leds_state[i]) {
            *(uint32_t*)(0x40010810) = (1UL << (5 + i)); // Включить лампу i
        } else {
            *(uint32_t*)(0x40010814) = (1UL << (5 + i));  // Выключить лампу i
        }
    }
}

void add_led_group(void) {
    // ВЫКЛЮЧАЕМ все лампы перед добавлением новой группы
    for (int i = 0; i < 6; i++) {
        leds_state[i] = 0;
    }

    // ВКЛЮЧАЕМ новую группу из led_count ламп, начиная с current_position
    for (int i = 0; i < led_count; i++) {
        int led_index = (current_position + i) % 6;
        leds_state[led_index] = 1; // Включаем лампу
    }

    // Обновляем позицию для следующей группы
    current_position = (current_position + led_count) % 6;
}
int main(void) {
    // Включаем тактирование порта A

    *(uint32_t*)(0x40021000 + 0x18) |= (1UL << 2UL); // IOPAEN = 1

    // Настраиваем пины PA5-PA10 как выходы push-pull 2MHz
    // MODE=10 (2 MHz), CNF=00 (push-pull) → 0x2
    // PA5-PA7 находятся в CRL, PA8-PA10 — в CRH
```

```
    // CRL: PA5, PA6, PA7
    // Очистка всей конфигурации для конкретных пинов
    *(uint32_t*)(0x40010800 + 0x00) &= ~((0x3UL << (20UL)) |
                                         (0x3UL << (22UL)) |
                                         (0x3UL << (24UL)) |
                                         (0x3UL << (26UL)) |
                                         (0x3UL << (28UL)) |
                                         (0x3UL << (30UL)));

    //Устанавливаем частоту PA5, PA6, PA7
    *(uint32_t*)(0x40010800) |=  ((0x2UL << (20UL)) |
                                  (0x2UL << (24UL)) |
                                  (0x2UL << (28UL)));

    // CRH: PA8, PA9, PA10
    *(uint32_t*)(0x40010804) &= ~((0x3UL << (0UL))  |
                                    (0x3UL << (2UL))  |
                                    (0x3UL << (4UL))  |
                                    (0x3UL << (6UL))  |
                                    (0x3UL << (8UL))  |
                                    (0x3UL << (10UL))) ;

    //Устанавливаем частоту 2 МГц
    *(uint32_t*)(0x40010804) |=  ((0x2UL << (0UL)) |
                                  (0x2UL << (4UL)) |
                                  (0x2UL << (8UL)));

    //Настраиваем кнопки PA0, PA1 как входы с подтяжкой вверх
    *(uint32_t*)(0x40010800) &= ~((0x3UL << (0UL)) |
                                  (0x3UL << (2UL)) |
                                  (0x3UL << (4UL)) |
                                  (0x3UL << (6UL)));

    *(uint32_t*)(0x40010800) |=  ((0x2UL << (2UL)) |
                                  (0x2UL << (6UL)));  // CNF=10, MODE=00



    *(uint32_t*)(0x4001080C) |= ((0x1UL << 0UL) | (0x1UL << 1UL)); // подтяжка
    //Подаём питание на при нажатии


    while (1) {
        uint8_t btn1 = (*(uint32_t*)(0x40010808) & (1UL << 0)) ? 1 : 0;
        uint8_t btn2 = (*(uint32_t*)(0x40010808) & (1UL << 1)) ? 1 : 0;

        // Кнопка 1: ТОЛЬКО задаёт количество одновременно включаемых светодиодов
        if (prev_btn1 == 1 && btn1 == 0) {
            led_count++;
            if (led_count > 6) led_count = 1;
            // Только меняем значение, не меняем состояние ламп
```

```
        }
        prev_btn1 = btn1;

        // Кнопка 2: добавляет следующую группу светодиодов
        if (prev_btn2 == 1 && btn2 == 0) {
            if (led_count == 6) {
                flag = 1;
                // если выбрано 6 — просто включаем/выключаем все
                int all_on = 1;
                for (int i = 0; i < 6; i++) {
                    if (!leds_state[i]) {
                        all_on = 0;
                        break;
                    }
                }

                if (all_on) {
                    // Выключаем все
                    for (int i = 0; i < 6; i++) {
                        leds_state[i] = 0;
                    }
                    current_position = 0;
                } else {
                    // Включаем все
                    for (int i = 0; i < 6; i++) {
                        leds_state[i] = 1;
                    }
                }
            } else {
                flag = 0;
                // Добавляем новую группу ламп (старые выключаются автоматически)
                add_led_group();
            }

            // Обновляем физические светодиоды
            update_physical_leds();
        }
        prev_btn2 = btn2;

        // Задержка для антидребезга
        for(uint32_t i = 0; i < 100000; i++);
    }
}
```

Define

1) Main.c

```
#include "Init.h"
#include "Init.c"

uint8_t flag;
// Переменные
int led_count = 1;      // сколько включается одновременно (1..6)
int current_position = 0; // текущая позиция для следующей группы
uint8_t prev_btn1 = 1; // предыдущее состояние кнопок
uint8_t prev_btn2 = 1;

// Массив состояний ламп (0-выкл, 1-вкл)
uint8_t leds_state[6] = {0, 0, 0, 0, 0, 0};



void update_physical_leds(void) {
    // Синхронизируем физическое состояние с логическим
    for (int i = 0; i < 6; i++) {
        if (leds_state[i]) {
            GPIO_PORT_A_BSRR = (1UL << (5 + i)); // Включить лампу i
        } else {
            GPIO_PORT_A_BRR = (1UL << (5 + i));  // Выключить лампу i
        }
    }
}

void add_led_group(void) {
    // ВЫКЛЮЧАЕМ все лампы перед добавлением новой группы
    for (int i = 0; i < 6; i++) {
        leds_state[i] = 0;
    }

    // ВКЛЮЧАЕМ новую группу из led_count ламп, начиная с current_position
    for (int i = 0; i < led_count; i++) {
        int led_index = (current_position + i) % 6;
        leds_state[led_index] = 1; // Включаем лампу
    }

    // Обновляем позицию для следующей группы
    current_position = (current_position + led_count) % 6;
}

int main(void) {
    GPIO_Ini();

    while (1) {
        uint8_t btn1 = (GPIO_PORT_A_IDR & (1UL << 0)) ? 1 : 0;
        uint8_t btn2 = (GPIO_PORT_A_IDR & (1UL << 1)) ? 1 : 0;
```

```
        // Кнопка 1: ТОЛЬКО задаёт количество одновременно включаемых светодиодов
        if (prev_btn1 == 1 && btn1 == 0) {
            led_count++;
            if (led_count > 6) led_count = 1;
            // Только меняем значение, не меняем состояние ламп
        }
        prev_btn1 = btn1;

        // Кнопка 2: добавляет следующую группу светодиодов
        if (prev_btn2 == 1 && btn2 == 0) {
            if (led_count == 6) {
                flag = 1;
                // если выбрано 6 — просто включаем/выключаем все
                int all_on = 1;
                for (int i = 0; i < 6; i++) {
                    if (!leds_state[i]) {
                        all_on = 0;
                        break;
                    }
                }

                if (all_on) {
                    // Выключаем все
                    for (int i = 0; i < 6; i++) {
                        leds_state[i] = 0;
                    }
                    current_position = 0;
                } else {
                    // Включаем все
                    for (int i = 0; i < 6; i++) {
                        leds_state[i] = 1;
                    }
                }
            } else {
                flag = 0;
                // Добавляем новую группу ламп (старые выключаются автоматически)
                add_led_group();
            }

            // Обновляем физические светодиоды
            update_physical_leds();
        }
        prev_btn2 = btn2;

        // Задержка для антидребезга
        for(uint32_t i = 0; i < 100000; i++);
    }
}
```

2) Init.c

```c
#include "init.h"

void GPIO_Ini(void){

    // Включаем тактирование порта A
    RCC_APB2ENR |= (IOPAEN); // IOPAEN = 1

    // Настраиваем пины PA5–PA10 как выходы push-pull 2MHz
    // MODE=10 (2 MHz), CNF=00 (push-pull) → 0x2
    // PA5–PA7 находятся в CRL, PA8–PA10 — в CRH
    // CRL: PA5, PA6, PA7
    // Очистка всей конфигурации для конкретных пинов
    GPIO_PORT_A_CRL &= ~(CRL_MODE_5 |
                         CRL_CNF_5  |
                         CRL_MODE_6 |
                         CRL_CNF_6  |
                         CRL_MODE_7 |
                         CRL_CNF_7) ;

    //Устанавливаем частоту PA5, PA6, PA7
    GPIO_PORT_A_CRL |=  (CRL_MODE5_2  |
                         CRL_MODE6_2  |
                         CRL_MODE7_2) ;

    // CRH: PA8, PA9, PA10
    GPIO_PORT_A_CRH &= ~(CRH_MODE_8  |
                         CRH_CNF_8   |
                         CRH_MODE_9  |
                         CRH_CNF_9   |
                         CRH_MODE_10 |
                         CRH_CNF_10) ;

    //Устанавливаем частоту 2 МГц
    GPIO_PORT_A_CRH |= (CRH_MODE8_2  |
                        CRH_MODE9_2  |
                        CRH_MODE10_2);

    //Настраиваем кнопки PA0, PA1 как входы с подтяжкой вверх
    GPIO_PORT_A_CRL &= ~(CRL_MODE0 |
                                  CRL_CNF0 |
                                  CRL_MODE1|
                                  CRL_CNF1);

    GPIO_PORT_A_CRL |=  (CRL_CNF0_1 |
                         CRL_CNF1_1);  // CNF=10, MODE=00
```

```
    GPIO_PORT_A_ODR |= (ODR0 | ODR1); // подтяжка
    //Подаём питание на при нажатии
}
```

3) Init.h

```
#include <stdint.h>

#define CRL (0x00)
#define CRH (0x04)
#define IDR (0x08)
#define ODR (0x0C)
#define BSRR (0x10)
#define BRR (0x14)

#define RCC_APB2ENR (*(uint32_t*)(0x40021000 + 0x18))
#define IOPAEN ((1UL << 2UL))

#define GPIO_PORT_A_CRL (*(uint32_t*)(0x40010800 + CRL))
#define GPIO_PORT_A_CRH (*(uint32_t*)(0x40010800 + CRH))
#define GPIO_PORT_A_IDR (*(uint32_t*)(0x40010800 + IDR))
#define GPIO_PORT_A_ODR (*(uint32_t*)(0x40010800 + ODR))
#define GPIO_PORT_A_BSRR (*(uint32_t*)(0x40010800 + BSRR))
#define GPIO_PORT_A_BRR (*(uint32_t*)(0x40010800 + BRR))

#define CRL_MODE_5 ((0x3UL << (20UL)))
#define CRL_MODE_6 ((0x3UL << (24UL)))
#define CRL_MODE_7 ((0x3UL << (28UL)))

#define CRL_CNF_5 ((0x3UL << (22UL)))
#define CRL_CNF_6 ((0x3UL << (26UL)))
#define CRL_CNF_7 ((0x3UL << (30UL)))

#define CRL_MODE5_2 ((0x2UL << (20UL)))
#define CRL_MODE6_2 ((0x2UL << (24UL)))
#define CRL_MODE7_2 ((0x2UL << (28UL)))

#define CRH_MODE_8 ((0x3UL << (0UL)))
#define CRH_MODE_9 ((0x3UL << (4UL)))
#define CRH_MODE_10 ((0x3UL << (8UL)))

#define CRH_CNF_8 ((0x3UL << (2UL)))
#define CRH_CNF_9 ((0x3UL << (6UL)))
#define CRH_CNF_10 ((0x3UL << (10UL)))

#define CRH_MODE8_2 ((0x2UL << (0UL)))
```

```
#define CRH_MODE9_2 ((0x2UL << (4UL)))
#define CRH_MODE10_2 ((0x2UL << (8UL)))

#define CRL_MODE0 ((0x3UL << (0UL)))
#define CRL_MODE1 ((0x3UL << (4UL)))

#define CRL_CNF0 ((0x3UL << (2UL)))
#define CRL_CNF1 ((0x3UL << (6UL)))

#define CRL_CNF0_1 ((0x2UL << (2UL)))
#define CRL_CNF1_1 ((0x2UL << (6UL)))

#define ODR0 ((0x1UL << 0UL))
#define ODR1 ((0x1UL << 1UL))

#define IDR0 ((0x1UL << 0UL))
#define IDR1 ((0x1UL << 1UL))


void GPIO_Ini(void);
```

CMSIS

1) подключаем соответствующие библиотеки

```
#include <stdint.h>
#include "../../CMSIS/Devices/STM32F1xx/Inc/STM32f1xx.h"
#include "../../CMSIS/Devices/STM32F1xx/Inc/STM32f103xB/stm32f103xb.h"
```

2) Включаем тактирование

```
SET_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPAEN);
```

3) Настраиваем пины PA5-PA10 как выходы push-pull 2МГц

MODE=10 (2МГц), CNF=00 (push-pull) ->0x2
PA5-PA7 находятся в CRL, PA8-PA10 находятся в CRH
Очищаем все конфигурации для пинов PA5-PA7

```
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_MODE5);
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_CNF5);

    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_MODE6_Msk);
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_CNF6_Msk);

    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_MODE7_Msk);
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_CNF7_Msk);
```

4) Устанавливаем частоту согласно Reference manual RM0008 равной 2 МГц

```
    SET_BIT(GPIOA->CRL, GPIO_CRL_MODE5_1);
    SET_BIT(GPIOA->CRL, GPIO_CRL_MODE6_1);
    SET_BIT(GPIOA->CRL, GPIO_CRL_MODE7_1);
```

5) Настраиваем пины PA8-PA10 как выходы push-pull 2МГц

Очищаем все конфигурации для пинов PA8-PA10

```
    CLEAR_BIT(GPIOA->CRH, GPIO_CRH_MODE8);
    CLEAR_BIT(GPIOA->CRH, GPIO_CRH_CNF8);

    CLEAR_BIT(GPIOA->CRH, GPIO_CRH_MODE9);
    CLEAR_BIT(GPIOA->CRH, GPIO_CRH_CNF9);

    CLEAR_BIT(GPIOA->CRH, GPIO_CRH_MODE10);
    CLEAR_BIT(GPIOA->CRH, GPIO_CRH_CNF10);
```

6) Устанавливаем частоту работы пинов равную 2МГц

```
    SET_BIT(GPIOA->CRH, GPIO_CRH_MODE8_1);
    SET_BIT(GPIOA->CRH, GPIO_CRH_MODE9_1);
```

```
    SET_BIT(GPIOA->CRH, GPIO_CRH_MODE10_1);
```

7) Настраиваем кнопки PA0, PA1 как входы с подтяжкой

Очищаем все конфигурации для пинов PA0, PA1

```
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_CNF0);
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_MODE0);
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_CNF1);
    CLEAR_BIT(GPIOA->CRL, GPIO_CRL_MODE1);
```

8)

```
    SET_BIT(GPIOA->CRL, GPIO_CRL_CNF0_1);
    SET_BIT(GPIOA->CRL, GPIO_CRL_CNF1_1);
```

9) Устанавливаем подачу питания при нажатии на кнопку

```
    SET_BIT(GPIOA->ODR, GPIO_ODR_ODR0);
    SET_BIT(GPIOA->ODR, GPIO_ODR_ODR1);
```

10) Вводим переменную для подсчёта включенных одновременно светодиодов для кнопки PA0

```
#include "Init.h"
#include "Init.c"

uint8_t flag;
// Переменные
int led_count = 1;      // сколько включается одновременно (1..6)
int current_position = 0; // текущая позиция для следующей группы
uint8_t prev_btn1 = 1; // предыдущее состояние кнопок
uint8_t prev_btn2 = 1;

// Массив состояний ламп (0-выкл, 1-вкл)
uint8_t leds_state[6] = {0, 0, 0, 0, 0, 0};

// Адреса регистров
#define GPIOA_IDR  (*(volatile uint32_t*)(0x40010808))
#define GPIOA_BSRR (*(volatile uint32_t*)(0x40010810))
#define GPIOA_BRR  (*(volatile uint32_t*)(0x40010814))

void update_physical_leds(void) {
    // Синхронизируем физическое состояние с логическим
    for (int i = 0; i < 6; i++) {
        if (leds_state[i]) {
            WRITE_REG(GPIOA->BSRR, 1UL << (5 + i)); // Включить лампу i
        } else {
            WRITE_REG(GPIOA->BRR, 1UL << (5 + i));  // Выключить лампу i
```

```
        }
    }
}

void add_led_group(void) {
    // ВЫКЛЮЧАЕМ все лампы перед добавлением новой группы
    for (int i = 0; i < 6; i++) {
        leds_state[i] = 0;
    }

    // ВКЛЮЧАЕМ новую группу из led_count ламп, начиная с current_position
    for (int i = 0; i < led_count; i++) {
        int led_index = (current_position + i) % 6;
        leds_state[led_index] = 1; // Включаем лампу
    }

    // Обновляем позицию для следующей группы
    current_position = (current_position + led_count) % 6;
}

int main(void) {
    GPIO_Ini();

    while (1) {
        uint8_t btn1 = READ_BIT(GPIOA->IDR, GPIO_IDR_IDR0)  ? 1 : 0;
        uint8_t btn2 = READ_BIT(GPIOA->IDR, GPIO_IDR_IDR1) ? 1 : 0;

        // Кнопка 1: ТОЛЬКО задаёт количество одновременно включаемых светодиодов
        if (prev_btn1 == 1 && btn1 == 0) {
            led_count++;
            if (led_count > 6) led_count = 1;
            // Только меняем значение, не меняем состояние ламп
        }
        prev_btn1 = btn1;

        // Кнопка 2: добавляет следующую группу светодиодов
        if (prev_btn2 == 1 && btn2 == 0) {
            if (led_count == 6) {
                flag = 1;
                // если выбрано 6 — просто включаем/выключаем все
                int all_on = 1;
                for (int i = 0; i < 6; i++) {
                    if (!leds_state[i]) {
                        all_on = 0;
                        break;
                    }
                }

                if (all_on) {
                    // Выключаем все
                    for (int i = 0; i < 6; i++) {
```

```
                        leds_state[i] = 0;
                    }
                    current_position = 0;
                } else {
                    // Включаем все
                    for (int i = 0; i < 6; i++) {
                        leds_state[i] = 1;
                    }
                }
            } else {
                flag = 0;
                // Добавляем новую группу ламп (старые выключаются автоматически)
                add_led_group();
            }

            // Обновляем физические светодиоды
            update_physical_leds();
        }
        prev_btn2 = btn2;

        // Задержка для антидребезга
        for(uint32_t i = 0; i < 100000; i++);
    }
}
```

Графики внешних воздействий

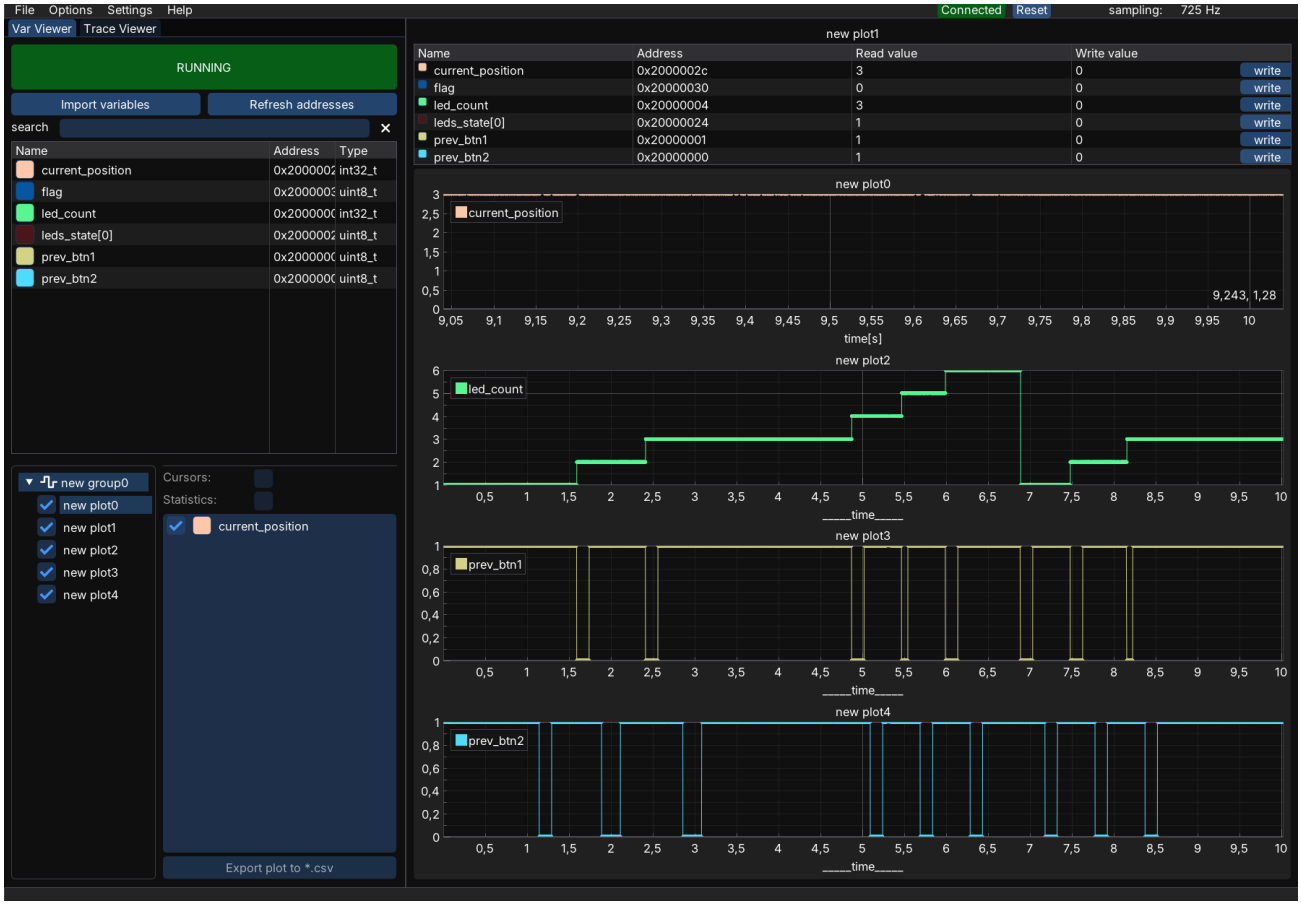


Рисунок 2 – Графики внешних воздействий

На рисунке 2 приведены графики внешних воздействий, где prev_btn1 отображает результаты нажатия на кнопку 1, которая устанавливает количество одновременно включаемых ламп, led count отображает количество включаемых ламп, prev_btn2 отображает количество нажатий на вторую кнопку, которая непосредственно включает светодиоды. Current position отображает текущую позицию, то есть запоминает номер последнего включенного светодиода.

Вывод

В ходе лабораторной работы создана и настроена среда разработки для программирования STM32 (на базе Visual Studio Code), выбраны и применены соответствующие плагины.

Собрана схема с шестью светодиодами и двумя кнопками, проведено подключение согласно заданию. Разработаны три подхода к управлению портами GPIO: прямой доступ по адресам, макросы и использование CMSIS.

Составлен и реализован алгоритм управления группами светодиодов с применением кнопок; алгоритм визуализирован в виде блок-схемы. Проведено тестирование и отладка с использованием MCUViewer — построены осциллограммы для контроля переходов состояний.

Программа работает корректно, логика переключения групп светодиодов выполняется по требованию:

а) кнопка 1 меняет количество одновременно горящих светодиодов (1–6).

б) кнопка 2 циклически сдвигает включённые светодиоды либо включает/выключает их все при числе 6.

Логика реализована и средствами низкоуровневого программирования, и через CMSIS.

Для подавления дребезга реализована простая задержка

Получено практическое понимание работы с портами ввода-вывода STM32, опробованы различные подходы к организации кода.

Томск 2025

Приложение А

Томск 2025

ФЮРА1 ЭЭ

STM32F103C8T6

Перв. примен.

Справ. №

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.		Степанов А.С.		
Пров.		Побережкин Н.И.		
Т.контр.				
Н.контр.				
Утв.				

ФЮРА1 ЭЭ

Принципиальная схема подключения

Лит.	Масса	Масштаб
		1:1
Лист	Листов 1	

ТПУ ИШИТР
Группа 8ЕМ51

Копировал

Формат А4